UCS503 SOFTWARE ENGINEERING PROJECT

Mid-Semester Evaluation Lab file

Submitted by:

102303855 - Guneev Taneja

102303874 - Manya Sharma

102303858 - Kinjal

102303844 - Mayukh Garg

BE Third Year, COE

Group No: 3C62

Submitted to: Dr. Raghav B.Venkatarmayier

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Computer Science and Engineering Department

TIET, Patiala

TABLE OF CONTENTS

1. PROJECT SELECTION PHASE

1.1 SOFTWARE BID

Software Bid/ Project Teams

UCS 503- Software Engineering Lab

Group : 3C62 Dated: 2nd November 2025

Team Name: KGM²
Team ID (will be assigned by Instructor):

Please enter the names of your Preferred Team Members. : Guneev Taneja, Manya Sharma, Kinjal, Mayukh Garg

• You are required to form **a three to four person** teams

• Choose your team members wisely. You will not be allowed to change teams.

Name	Roll No	Project Experience	Programming Language used	Signature
Guneev Taneja	102303855	1) Personal budget tracker 2) Movie recommendation system	Python, HTML, CSS, JS	Guneev
Manya Sharma	102303876	1) Online job portal 2) Music recommendation system	Python, HTLM, CSS, JS	Manya
Kinjal	102303858	1) Leave management system, 2) Movie recommendation system	Python, HTML, CSS, JS	Kinjal
Mayukh Garg	102303844	1) Tourist place recommendation system 2) Social media friend suggestion system	Python, HTML, CSS, JS	Mayukh

Programming Language / Environment Experience

List the languages you are most comfortable developing in, **as a team**, in your order of preference. Many of the projects involve Java or C/C++ programming.

1. Python
2. SQL
3. C++

Choices of Projects:

Please select **4 projects** your team would like to work on, by order of preference: *[Write at-least one paragraph for each choice (motivation, reason for choice, feasibility analysis, etc.)]*

	Project Name	Unique Selling Point
First Choice	Recall AI- AI-powered second brain that passively tracks your digital life and autogenerates daily memory scrolls.	• Passive Memory AI • Context-Aware Summaries • Smart Recall for ADHD • Zero input, total digital recall.

Second Choice	DigiWise- Personal Financial Analytics Dashboar	• Predictive Budgeting • Auto Categorization •Anomaly detection for overspending
Third Choice	MindMesh: Real-time Mental Health Detector & Scheduler	• Multimodal XAI-backed stress detection • Smart auto-planning
Fourth Choice	LawBot AI: Legal Document Analyzer with Chat Interface	• Legal awareness • LLM-based Q&A • Risk flagging

Additional Remarks/ Inputs

Please tell us about any other factors that we should take into consideration (e.g., if you really would like to work on a project for some particularly convincing reason).
Project name: **Recall AI**

- **Project for Resume**
- **Feasibility (Team Skills)**
- **React for frontend, AI/ML for summarization, and API/backend (Supabase, FastAPI, OpenAI).**
- **Unique Selling Points:**
    1. Zero-input memory scrolls from digital activity.
    2. Built for ADHD/neurodivergent users, not just general productivity.
    3. Smart recall & anomaly detection across platforms.

1.2 PROJECT OVERVIEW

1. Project Title:

Recall AI – Smart Meeting Memory & Note Assistant

2. Objective:

The main goal of Recall AI is to automate the process of capturing, summarizing, and recalling meeting information using Artificial Intelligence.
It acts as a virtual memory assistant that listens to conversations or meetings, extracts key insights, and provides smart summaries, task lists, and reminders for users and teams.

3. Problem Statement:

In modern workplaces, people attend multiple meetings daily and often forget key points, action items, or context from past discussions.
Manual note-taking and searching through recordings are **time-consuming** and **inefficient**.
Recall AI solves this by providing an **AI-powered system that automatically records, processes, and retrieves meeting insights** when needed.

4. Key Features:

- **Automatic Speech-to-Text Conversion:** Converts meeting audio into accurate transcripts.
- **AI Summarization Engine:** Generates short, context-rich summaries highlighting key points and decisions.
- **Task Extraction:** Identifies and lists action items or commitments discussed during meetings.
- **Smart Recall:** Allows users to query past meetings via natural language (e.g., "What did we decide about project deadlines last week?").
- **Notification Service:** Sends reminders or follow-ups based on extracted tasks.
- **Secure Data Storage:** All audio and text data are stored securely with user authentication and access control.

5. System Architecture:

The project is structured into several modular components:

- **Frontend Interface:**
  Built with **ReactJS** or **Flask templates**, providing users with dashboards to upload recordings, view summaries, and search meeting notes.
- **Backend Engine:**
  Developed in **Python (Flask or FastAPI)**, handling API requests, model inference, and data flow.

- **AI Models Used:**
  - **Speech Recognition:** Whisper / Google Speech-to-Text
  - **Summarization & NLP:** OpenAI GPT model or BERT-based summarizer
  - **Task Extraction:** Custom fine-tuned NLP model using spaCy or transformers
- **Database:**
  Uses **MongoDB** or **PostgreSQL** to store user data, transcripts, and summaries.
- **Authentication Module:**
  Managed by **JWT tokens** or **OAuth2.0**, ensuring secure access.
- **Notification & Reminder Service:**
  Built with background schedulers (Celery or APScheduler) integrated with email or push notifications.

6. Workflow Overview:

1. **User uploads or records a meeting audio.**
2. **System transcribes** the audio into text using speech recognition.
3. **NLP pipeline analyzes** the transcript to extract key points, tasks, and decisions.
4. **Summarized report** and **task list** are generated and stored.
5. **User queries** Recall AI later to retrieve specific insights from past meetings.

7. Use Case Examples:

- A **student team** uses Recall AI to summarize group project discussions.
- A **corporate team** retrieves decisions from a past meeting without rewatching the recording.
- A **manager** automatically receives a task list via email after every meeting.

8. Technologies Used:

Category	Tools / Frameworks / Technologies
Frontend	HTML, CSS, JavaScript, ReactJS / Flask Templates
Backend	Python, Flask / FastAPI
AI & NLP Models	OpenAI GPT (for summarization), Whisper (for speech-to-text), BERT, spaCy
Database	MongoDB / PostgreSQL

Authentication	JWT (JSON Web Tokens), OAuth 2.0
Notification & Scheduling	Celery, APScheduler, SMTP / Push Notifications
Deployment & Hosting	Render, Heroku, or Local Server
Version Control	Git & GitHub
Development Tools	VS Code, Postman, Anaconda / Virtual Environment
Testing & Debugging	Pytest, Postman API Testing, Logging Module

9. Expected Outcomes:

- **Increased meeting productivity** by eliminating manual note-taking.
- **Accurate AI-based recall** of past information and decisions.
- **User-friendly dashboard** for reviewing transcripts, summaries, and tasks.
- **Scalable architecture** for multi-user environments (teams, organizations).

10. Future Enhancements:

- Integration with **Zoom, Google Meet, and Microsoft Teams**.
- **Voice-controlled recall queries** using voice assistants.
- **Multilingual transcription and summarization.**
- **Emotion analysis** to understand meeting tone and engagement.
- **Team collaboration features** for shared notes and summaries.

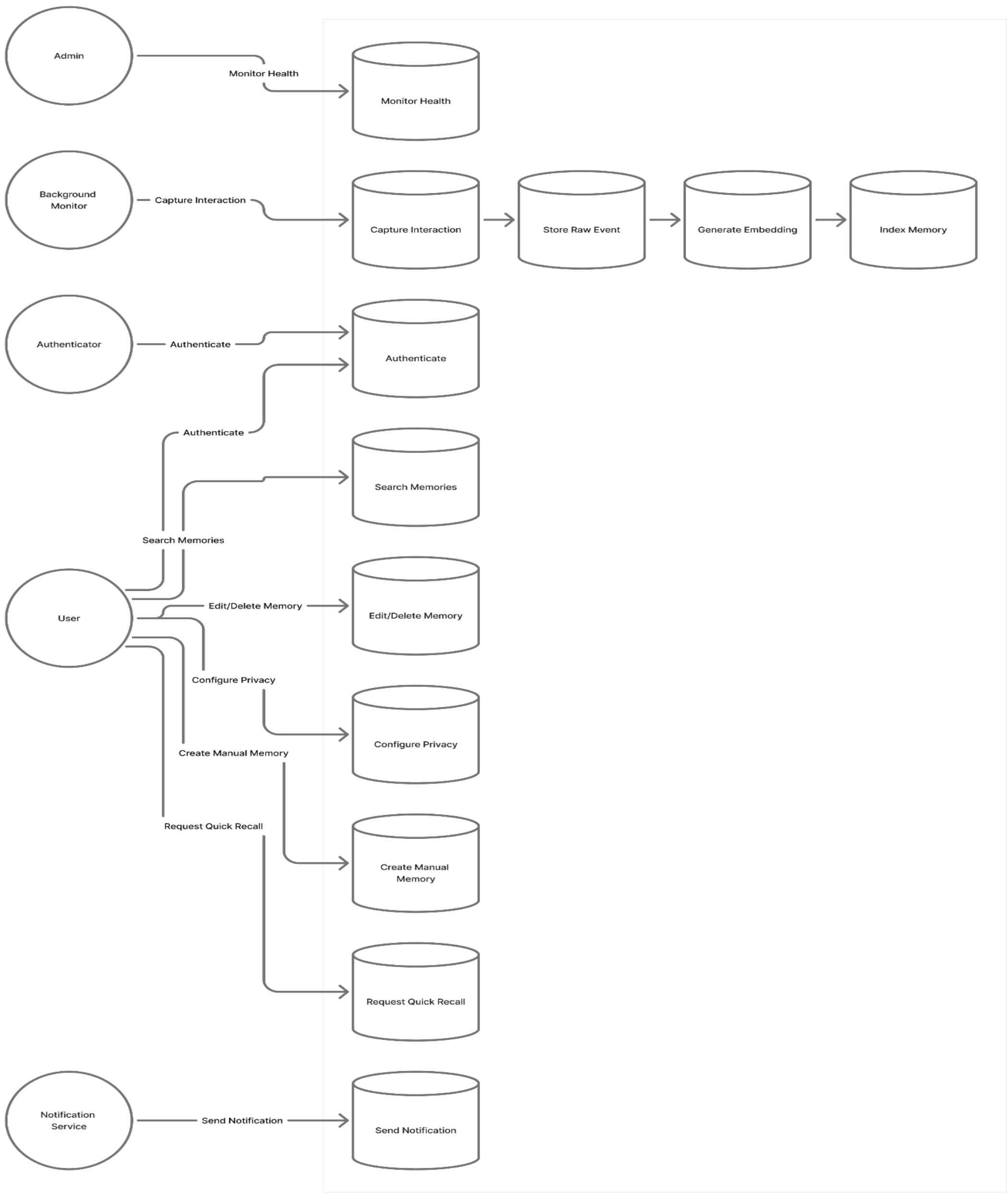
11. Project Status:

- Prototype completed with transcription and summarization modules.
- Integration of **task extraction and reminder services** in progress.
- Next step: **Deploying on a cloud server** and improving query accuracy.

2. ANALYSIS PHASE

2.1 USE CASES

2.1.1 USE-CASE DIAGRAM



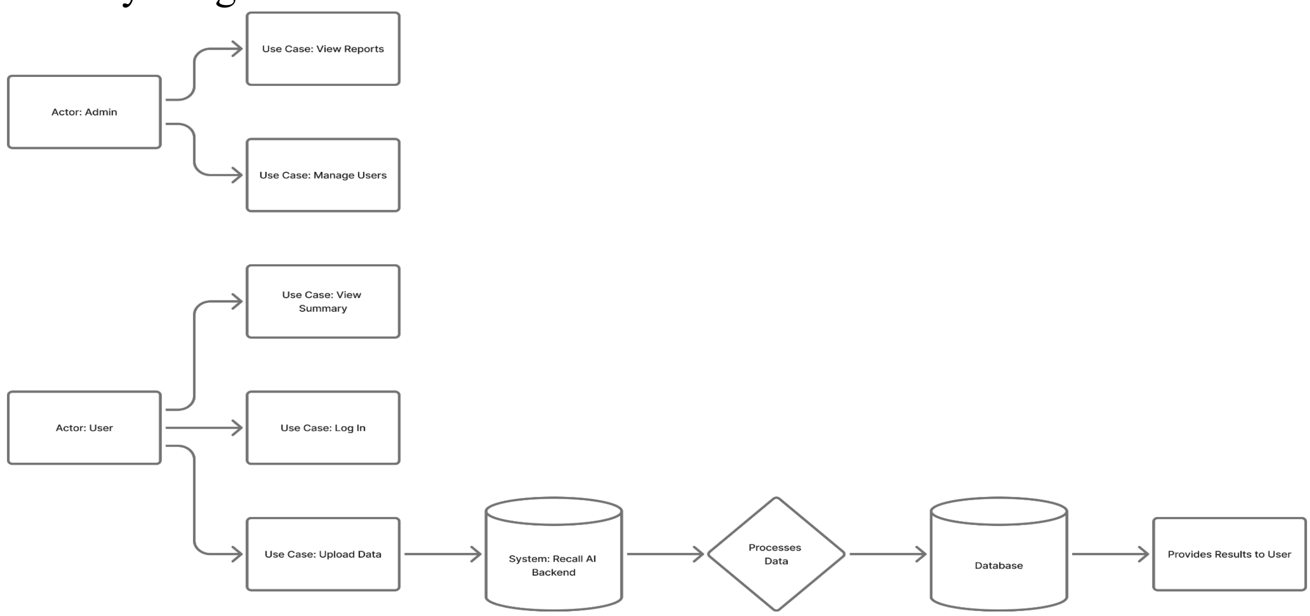
2.1.2 USE CASE TEMPLATE

Use Case ID	Details	Flow & Notes
UC-001 High **Title: Capture Interaction** (automa Actors: ,, a, HNN Priority: High Trigger: Monitor authenicated Post-cond: Raw event stored	Main Scenario 1. Detect event 2. Filliœ 4 Offline cached Exceptiove Irval palyad (4xx) NFRS: 2s ledaction	Main Scenario 1. Detect event 3. Send F:aching, 4. Privacy toggle Exeqticng: Embteding failure (retry) Acceptance: Events captured & searcible
UC-002 High **Generate Daily Summary** . High Trigger: Nightly schedule Post-cond: Sumnary created	Main Scenario 1. Select memories 3. Store 4 5. Pesults created	Main Scenario 1. KNN search for terank 4. Present results Acceptence: Summance Pfilch mairtems
UC-003 High **Senerate Memories (semantic)** . High Trigger: Events available Post-cond: Results returned	Main Scenario 1. Entie query 2. KNN search 3. Rerank limits Allceptiave: Summary reflects main items	NFRS ... Minutes to generate, ≤ 200 words 4. Display answed
UC-003 Med **Quick Recall (keyboard shrotut)** . High Keyboard schrocut Trigger: User query	Main Scenario 1. Shorcut, Query modal NRS: <2s response (best-effert)	Main Scenario 2. Query re:pmss (best-effert) Acceptence: Fast & relevance (precision > X)
UC-004 Med **Create Manual Memory** s, HIN Trigger: User logged in Trigger: Memory saved	User inputs text/media 2. Tags/context added 2 3. Save 4. Embex Fast & Unir blsd & inded	Main Scenario 1. User inputs text/media 2 3. Smed & index 4. Update update/removed
UC-006 Med **Edit/Delete Memory** . High Trigger: User owns mencuit Post-cond: Memory sarinned	Security: Privacy checks, audit log 2. Tags/context added 3 Acceptance: Fast & relevant arimted Memory searacible	Sotar auer selects memory 2. Edit/Delete 3. Confim 4. Confiim 4. Update/Purge DB (optional) Acceptence reflected, delestion confirmed

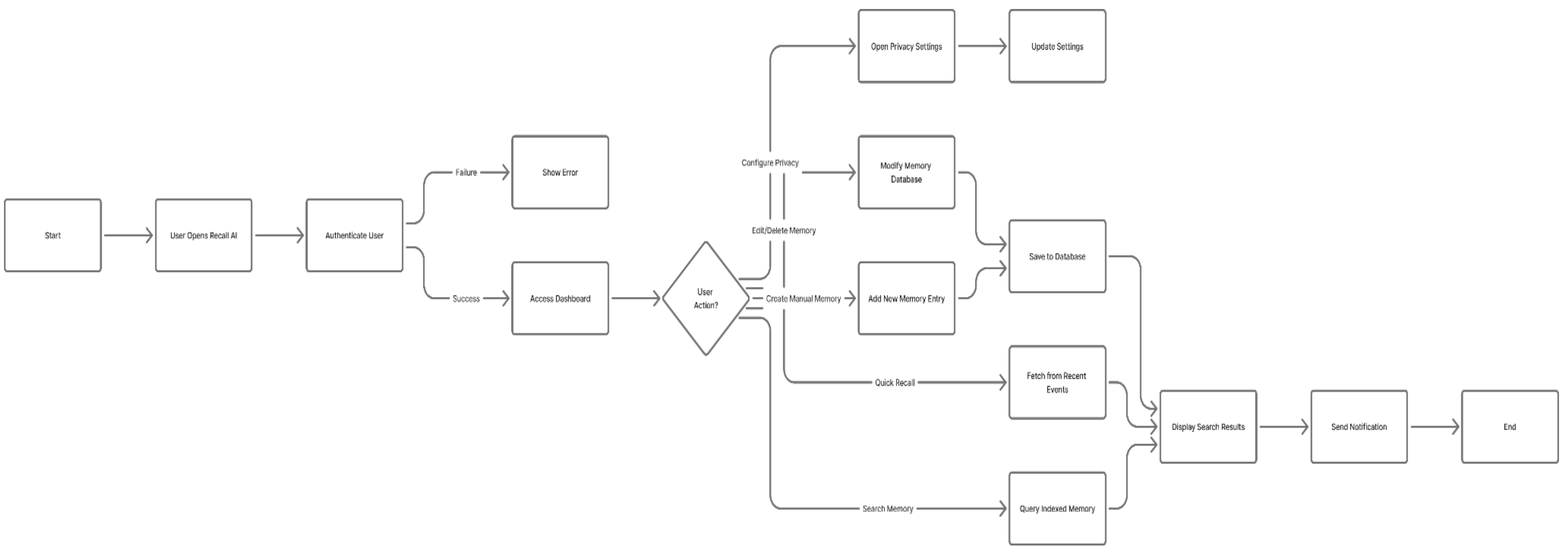
Version 1.0 - Recall AI Use Cases

2.2 Activity Diagram and Swimlane Diagrams

a) Activity Diagram:

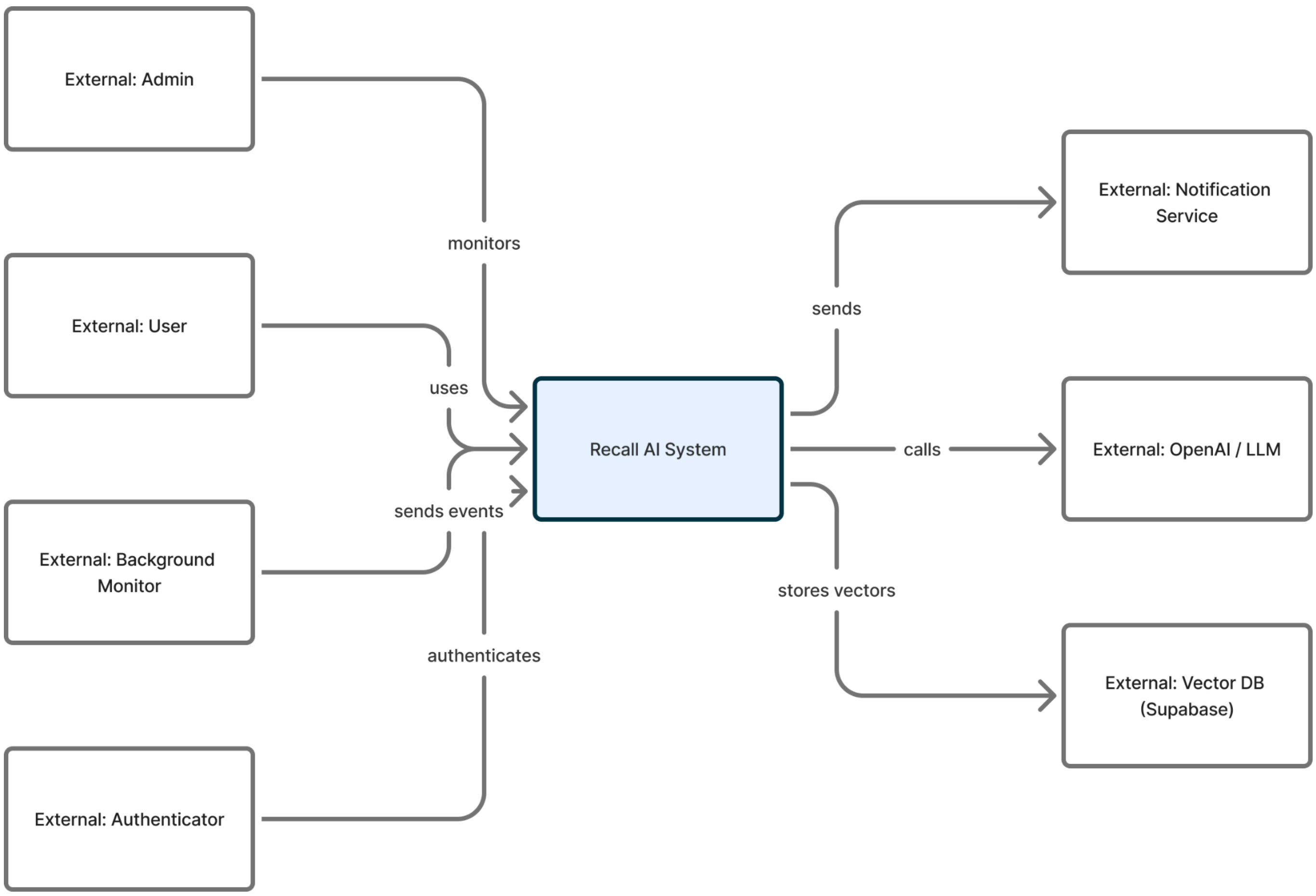


b) Swimlane Diagram:

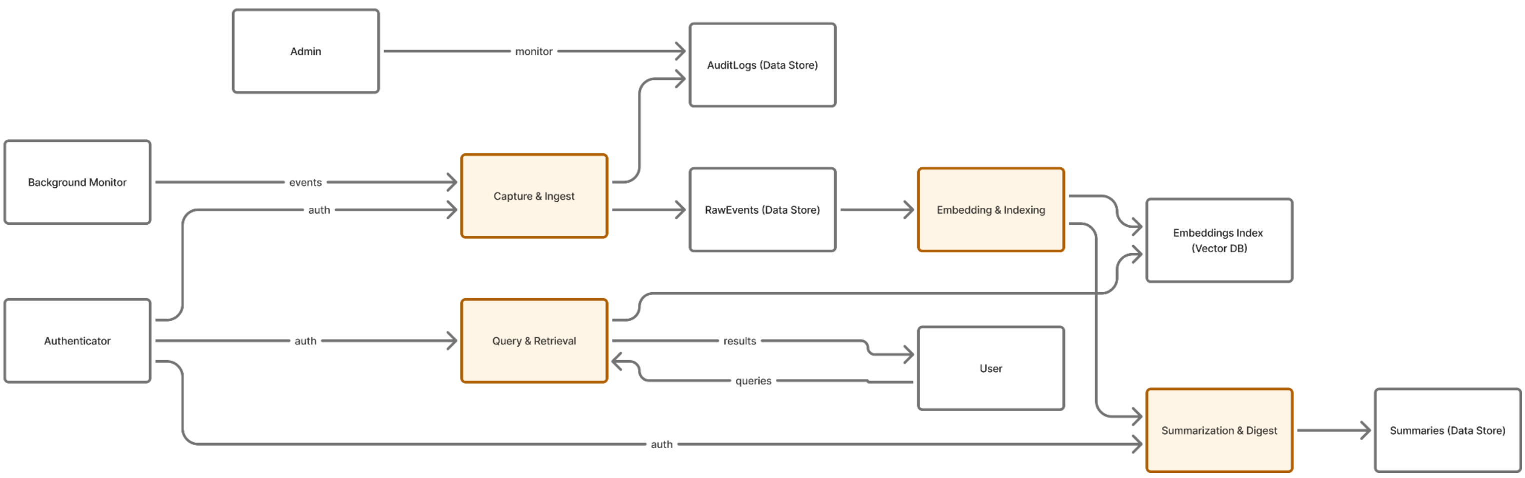


2.3 Data Flow Diagrams (DFDs)

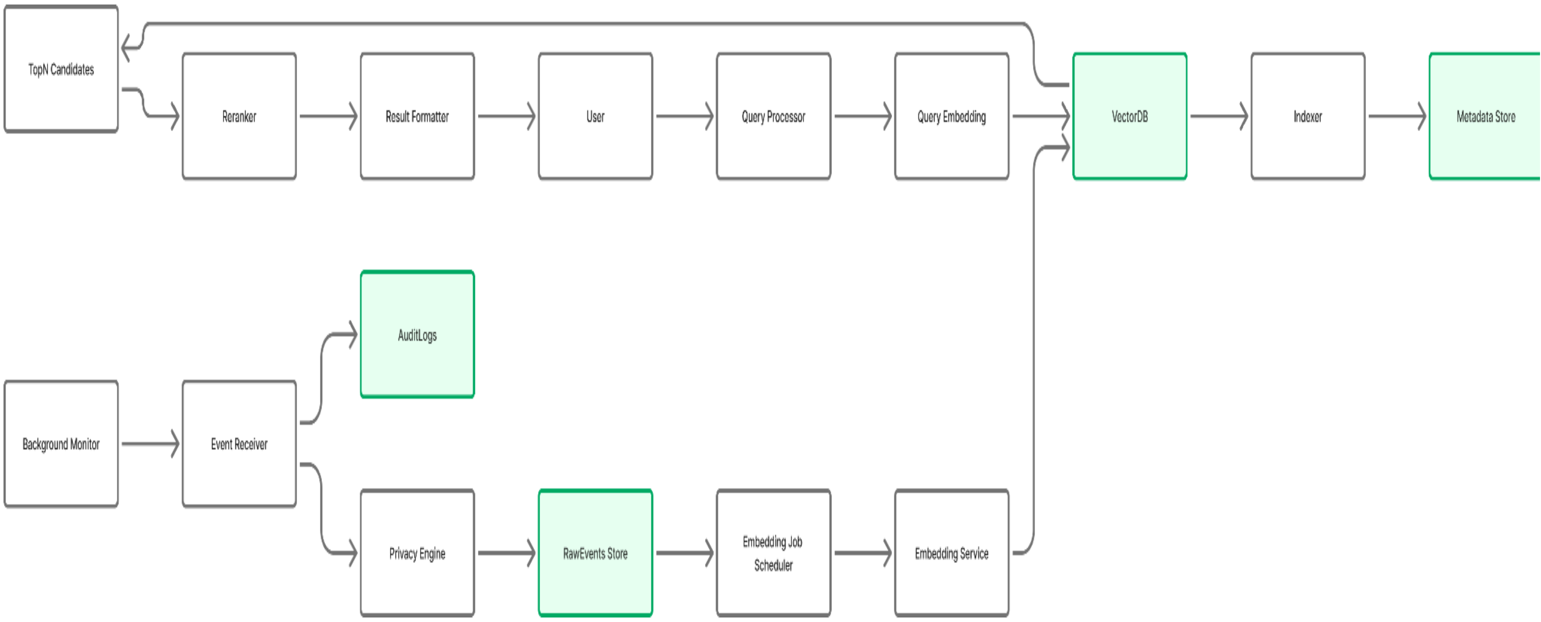
2.3.1 DFD Level 0



2.3.2 DFD Level 1



2.3.2 DFD Level 2



2.4 SOFTWARE REQUIREMENT SPECIFICATION IN IEEE FORMAT

- **Title: Recall AI — Memory Extension for ADHD and Neurodivergent Users**
- **Author: KGM²**
- **Date: 2nd December, 2025**
- **Version: 1.0**

1. Introduction

1.1 Purpose

This SRS describes functional and non-functional requirements for Recall AI — a system that passively captures a user's digital interactions, generates daily memory summaries, and provides semantic recall/search capabilities to aid users with memory and organization.

1.2 Scope

Recall AI provides passive capture (via background monitor), secure storage, vector-indexed memory retrieval, daily summarization, user controls for privacy/consent, and cross-device synchronization.

1.3 Definitions, Acronyms, Abbreviations

- LLM: Large Language Model
- KNN: K-nearest neighbors
- **SRS: Software Requirements Specification**

1.4 References

1. OpenAI API Documentation – *Used for implementing AI-based text understanding and embedding generation.* https://platform.openai.com/docs
2. Supabase Documentation – *Used for authentication, database management, and API integration.* https://supabase.com/docs
3. IEEE Software Engineering Standards (IEEE 830-1998) – *Referenced for requirements specification formatting and structure.*
4. GDPR (General Data Protection Regulation) – *Referenced for user data privacy and compliance guidelines.* https://gdpr.eu/
5. ISO/IEC 27001:2022 – Information Security Management – *For ensuring data protection and secure system design.*

6. PlantUML Documentation – *Used for creating UML diagrams and visual models.* https://plantuml.com/

2. Overall Description

2.1 Product Perspective

System is cloud-native with client-side monitors (extensions/agents), backend services (API), vector DB, OpenAI for LLM tasks, and UI clients.

2.2 Product Functions

- Event capture & filtration
- Embedding & indexing
- Semantic search & quick recall
- Summarization & digests
- Privacy controls & export
- Notifications & reminders

2.3 User Characteristics

Primary users are neurodivergent individuals seeking memory aids; assume varying tech-savviness. Accessibility required (keyboard nav, screen-reader support).

2.4 Constraints

- Must comply with GDPR-like privacy requirements where applicable.
- Token limits from LLM provider.
- Offline caching required.

2.5 Assumptions & Dependencies

- Users will permit installing a monitor on their devices.
- Availability of OpenAI API and Supabase or equivalent vector DB.

3. System Features and Requirements

3.1 Capture Interaction

- FR1.1: System shall accept event payloads from monitors (fields: user_id, timestamp, source_type, content_url, snippet, metadata).

- FR1.2: System shall apply privacy rules before storing.
- Acceptance: Test payloads from monitor appear in DB and are indexed.

3.2 Embedding & Indexing

- FR2.1: System shall enqueue embedding jobs upon storing raw events.
- FR2.2: System shall persist vectors with metadata for retrieval.
- NFR: Embedding throughput >= X events/minute; retries on failure.

3.3 Semantic Search

- FR3.1: System shall accept NL queries and return top-N relevant memories.
- FR3.2: System shall support reranking using contextual LLM calls.
- Acceptance: Precision and recall meet target thresholds on a test dataset.

3.4 Summaries

- FR4.1: System shall generate daily summaries using LLM.
- FR4.2: Summaries must include 3–5 highlights and links to original items.
- NFR: Completion latency ≤ Y minutes.

3.5 Privacy & Consent

- FR5.1: System shall allow user to configure domains/apps to exclude.
- FR5.2: System shall allow export & deletion of user data.
- Security: Use encrypted at-rest storage for sensitive fields.

3.6 Authentication & Authorization

- FR6.1: OAuth2 / token-based authentication.
- FR6.2: Short-lived tokens for monitors.

4. External Interface Requirements

- User interfaces: Web UI + mobile PWA screens (list of main screens).
- APIs: REST endpoints for ingest (/events), search (/search), summaries (/summaries), auth (/auth), privacy (/privacy).
- Hardware Interfaces: Local monitor requires minimal CPU/bandwidth.

5. Nonfunctional Requirements

- Performance: Typical search < 500ms for vector DB KNN (subject to infra).
- Reliability: 99.5% uptime for core services.
- Scalability: Horizontal scaling for embedding workers.
- Security: TLS, encryption at rest, audit logs, data deletion.
- Privacy: Default opt-out of capturing sensitive domains; PII redaction.

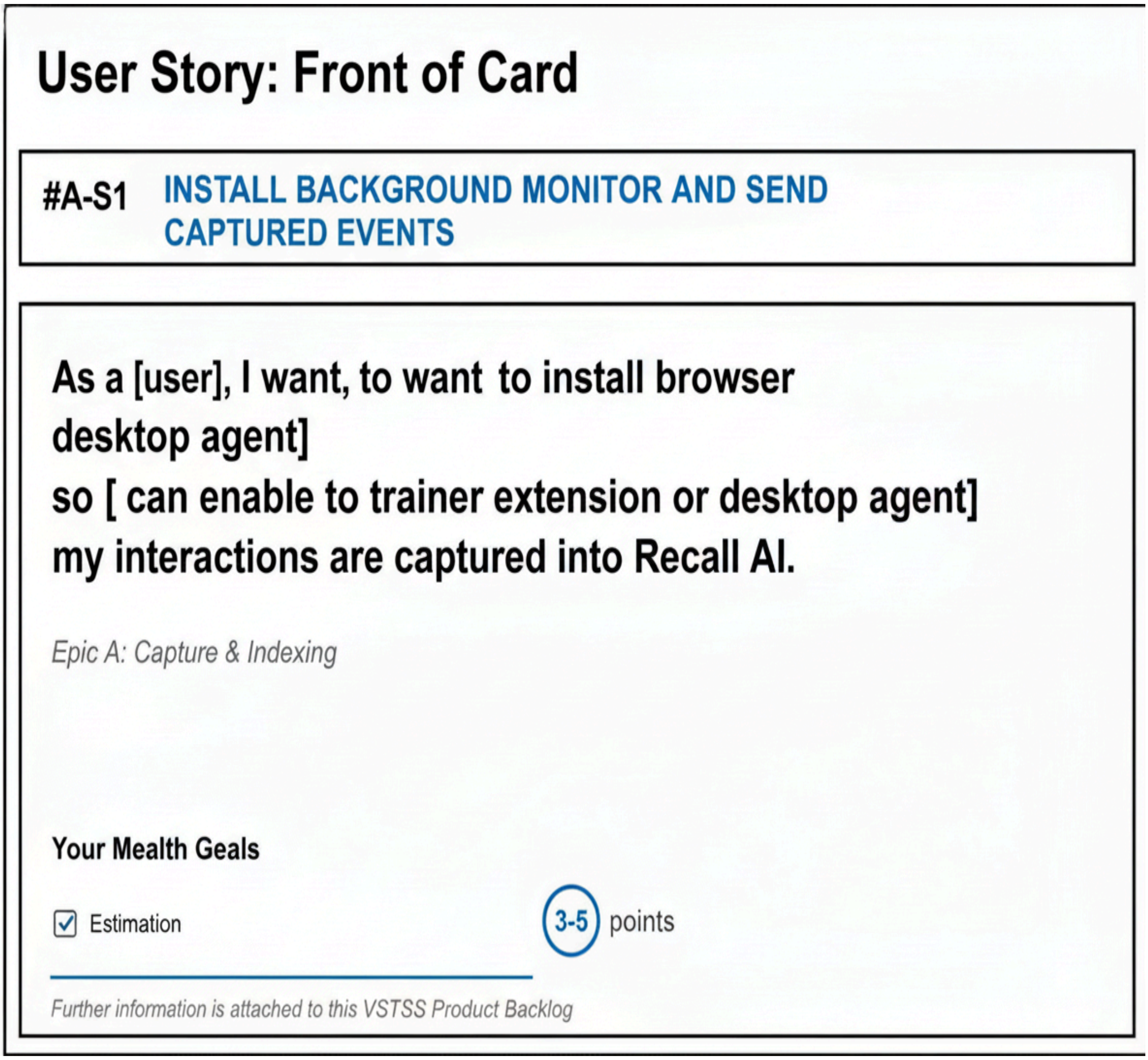
6. Other Requirements

- Internationalization: English-first; provide hooks for other languages.
- Accessibility: WCAG AA compliance for UI.

Appendices

- Data model diagrams, API schemas (OpenAPI), sample payloads.

2.5 USER STORIES AND USER CARDS



#A-S1 Story Back of Card

INSTALL BACKGROUND MONITOR AND SEND CAPTURED EVENTS

Confirmation Scenarios

Successful Install: Monitor extension/agent shows "Connected" status.
OaAth Authentication: User redrazed auccess ao succes, token stored securely.
Test Event Sent: New entriess show ingestion with test data.
Tata Appears: no Backend visgs show visible "RawEvents" table in database.
Data Appears: New entible ingetios forah database.
Error handling for failed processing or low latetase.

Technical Notes

Oaath 2.0 flow for secure authentication.
Endpont: /apievents acsur authentication.
Endpont: /apievents' accpts POST requests JSON palyload.
Data schema valiation fon fo for incoming events.
Database: Insert into 'RawEvens' table with tiamsstapm.
Error handling for failed connection for daw latency.

Test Cases

TC-1: Install & Authonicate

Steps: User adds extension/installs agent → Clicks "Complets OAh flow.
Expected: Status shows shows "Comcetied, network tab", auth token presnt.

TC-2: Send Test Event

TC-3: Backend Data Verifiation

Steps: Check "Send Tetent in database – Expected chew POST to witih 5 secds.
Expected New offisternet – Send test test event.
Steps: UI Disconet with tith error mesage, no data in no data in backend.

Further information is attached to this VSTSS Product Backblog